

Git, GitHub & Streamlit

Python Level 3 – Advanced course

Class 3 – Modules and Packages

Fábio Neves-Moreira

Python instructor



TWO TOOLS YOU WILL HEAR ABOUT

For information – nothing to install, nothing to run

These two names come up whenever people talk about a "real" Python project. By the end of this class you will know what they mean.

Why there is no exercise: both work on files on your own computer, through an editor such as VS Code. Colab has neither, so using them properly is outside this course.

What you are expected to take away: the vocabulary, and a clear idea of what each one is for – so that "push it to the repo" and "build a Streamlit app" stop being noise.

Where they fit: this is the natural next step after the course, not part of it.



VERSION CONTROL



THE PROBLEM IT SOLVES

Everyone has done this

```
report.ipynb  
report_v2.ipynb  
report_final.ipynb  
report_final_ANA.ipynb  
report_final_USE_THIS.ipynb
```

It works – until someone asks what changed between two of them.

Or until two people edit at the same time and one copy quietly wins.

Or until you need the version from three weeks ago, and it is gone.

Version control keeps every version of a project, who changed what, and why – without the filenames.

WHAT VERSION CONTROL DOES

Three things you cannot get from saving copies

Keeps the history. Every saved version stays, each with a note saying what changed and why. You can read back through it, and you can go back to any point.

Lets two people work at once. Each person works on their own copy. The tool then combines both sets of changes, instead of one overwriting the other.

Makes the project shareable. One shared copy that everybody works from, and that anybody new can take a copy of.

THE VOCABULARY

Six words – this is the slide worth remembering

Repository ("repo") – a project the tool is watching. A folder, plus its history.

Commit – a saved point, with a short message saying what changed. Your unit of work.

Push – send your commits to the shared copy, so others can see them.

Pull – bring everyone else's changes down into yours.

Branch – a side line of work, so unfinished changes stay out of everyone's way.

Merge – bring a branch's work back into the main line.

So "push it to the repo" means: save your work with a note, and send it to the shared copy.

GIT AND GITHUB ARE NOT THE SAME THING

One is the tool, the other is the website

Git – the tool itself.

Runs on your own computer. It is what actually records the history.

It works perfectly well with no internet and no account, on a project only you will ever see.

GitHub – a website that hosts repositories.

Where the shared copy lives, so a team can reach it.

Adds the things a team needs around the code: reviews, issues, and a page anyone can read.

Git is the engine. GitHub is one place to park it – there are others. You can use Git without ever touching GitHub.

A DAY'S WORK, IN FOUR COMMANDS

You will see these written down everywhere

```
git pull

# ... do your work ...

git add report.py
git commit -m "Add monthly totals"
git push
```

pull – start from what everyone else has already done.

add – choose which changed files belong in this saved point.

commit – save that point, with the message that explains it.

push – send it to the shared copy.

That is the whole loop, repeated all day. The rest of Git is what you learn when something goes wrong.

WHERE YOU HAVE ALREADY MET IT

Whether or not you have used it

Every library you import. pandas, matplotlib, seaborn and Streamlit are all developed on GitHub, in the open. You can read the source of `pandas.read_csv` – it is a public page.

Their history is public too. You can see when a bug was fixed, who fixed it, and what they changed. That is often the fastest way to understand behaviour that surprises you.

This course. These notebooks are kept in a repository. That is how a correction reaches you without anyone emailing a new file.

WHY WE ARE NOT USING IT

An honest answer

Git works on files in folders on your computer, and it is driven from a command line or an editor's interface.

Colab gives you neither. It gives you one notebook, in the browser, saved to Drive – which is exactly what makes it a good place to learn Python, and a poor place to learn Git.

The step that unlocks it: install Python and VS Code on your own machine. Everything you have written in this course runs there unchanged – and Git is waiting for you when it does.

Nothing you have learned changes. You just get more places to put it.



STREAMLIT



THE PROBLEM IT SOLVES

You built something useful. Now what?

You have written the expenses review: it loads the file, cleans it, groups by department and draws the chart. It answers a question your colleagues actually ask.

To let one of them use it today, you would have to send them a notebook, get them into Colab, explain which cell to run and where to change the department. Most people will not do that. So you end up running it for them, every time they ask.

Streamlit turns the same script into a web page. They open a link, choose what they want from a dropdown, and read the answer. They never see Python.

WHAT STREAMLIT IS

A Python library, not a new language

It is just a library. `pip install streamlit`, then import it like any other.

You write ordinary Python. The same pandas, the same matplotlib, top to bottom, exactly as in a notebook.

Each st. line puts one thing on the page. A title, a dropdown, a table, a chart. One line, one thing.

There is no HTML, CSS or JavaScript. That is the entire reason it exists – the web part is handled for you.

WHAT THE CODE LOOKS LIKE

Read it once – there is nothing new in it

```
import streamlit as st
import pandas as pd

expenses = pd.read_csv("expenses.csv")

st.title("Expenses review")

chosen = st.selectbox(
    "Department",
    sorted(expenses["department"].unique()),
)

shown = expenses[
    expenses["department"] == chosen
]
st.dataframe(shown)
```

Only the four st. lines are new. Everything else is Level 1 Class 3, unchanged.

st.title puts a heading on the page.

st.selectbox draws a dropdown – and hands back whatever the viewer picked, as an ordinary Python string.

st.dataframe shows a DataFrame as a sortable table.

The filtering in between is the boolean filtering you already write.

THE ONE IDEA WORTH REMEMBERING

How a page can be interactive with no extra work from you

When the viewer changes the dropdown, Streamlit runs your whole script again, from the first line to the last, with the new value.

That is all. There is no event handling to write, no callbacks, nothing to wire up.

Which is why it feels like a notebook: your script is still a straight line, read top to bottom. It simply gets read again each time somebody touches a control.

One script. Re-run on every interaction. That single sentence explains almost everything about how Streamlit behaves.

WHAT YOU CAN PUT ON A PAGE

A small vocabulary covers most of it

Text – `st.title`, `st.header`, `st.write`. `st.write` takes almost anything and shows it sensibly.

Controls – `st.selectbox`, `st.slider`, `st.checkbox`, `st.text_input`. Each returns the value the viewer chose.

Data – `st.dataframe` for a table, `st.metric` for a single number, `st.line_chart` and `st.bar_chart` for a quick chart.

Your own charts – `st.pyplot` puts a matplotlib figure on the page, so the charts from Level 2 Class 3 work as they are.

Layout – `st.sidebar`, `st.columns`, `st.expander` to arrange it.

THE DEMONSTRATION

expenses_app.py – the same data you have used since Level 1

Watch the dropdown. Every number, the chart and the table all change together – because the whole script ran again.

Watch the script beside the page. `st.title` is the heading, `st.selectbox` is the dropdown, `st.metric` is a number, `st.dataframe` is the table.

Look at where the data is loaded. That is the Level 1 Class 3 cleaning, unchanged. The chart is Level 2 Class 3 matplotlib, unchanged.

The honest summary: you have already written most of this app. The only new part is the `st.` lines.

WHY WE ARE NOT USING IT EITHER

And what it would take

Streamlit starts a small web server on your machine and serves a page from it. It is run from a terminal – streamlit run app.py – against a .py file, not a notebook.

Colab cannot do that: there is no terminal you keep open, and no page it can serve to you.

What it would take: Python and VS Code installed locally, pip install streamlit, and the file we just looked at. That is genuinely the whole list.

And then: put the project on GitHub and Streamlit will host it for free, at a link you can send to anyone. That is why these two topics share a slide deck.

WHAT TO TAKE AWAY

Three sentences

Git keeps the history of a project and lets several people work on it at once. GitHub is where the shared copy lives.

Streamlit turns a Python script into a web page, by re-running the script every time somebody changes a control.

Both need a local editor such as VS Code – which is exactly where to go after this course. Everything you have learned here works there, unchanged.

You have finished Level 3. The last class is your project – modules, classes and pandas, put to work on one problem.



THANK YOU 😊

Questions?

Fábio Neves-Moreira



Python Level 3 - Class 3